LAB EXERCISES 05 & 06

VulnWatch: React CRUD Application

DOMAIN / CYBERSECURITY TECH / REACT · JSX · VITE · LOCAL STORAGE · JSON SERVER

01 — Objective

Convert the prior VulnWatch HTML/CSS/JavaScript concept (Labs 2–4) into a component-based React application and extend its software watchlist to support persistent Create, Read, Update and Delete (CRUD) operations.

02 — Proposed solution

The updated VulnWatch retains its navy/blue cyber-intelligence aesthetic while adding a working software inventory workspace. Users register a technology, set its environment and monitoring priority, update or delete the entry, and search and filter the watchlist. The interface recalculates summary metrics as records change.

03 — React architecture

MODULE	RESPONSIBILITY
App.jsx	Owns records, persistence status, CRUD handlers and notifications
SoftwareForm + Watchlist	Controlled inputs, validation, create/read/update/delete and search
SignalFeed + PriorityLab	Dynamic scenario matching, severity filtering and slider-based model
LocalRadar + Hero + SummaryMetrics	Browser geolocation, overview presentation and computed metrics
records.js + storage.js	Reusable rules and Local Storage / REST data adapters

04 — Lab 5: React requirements

Implemented: functional components, JSX, useState, useEffect, event handling, reusable UI, controlled multi-field forms, radio-button priority selection, dropdowns, checkbox monitoring state, search/filter inputs and responsive CSS. React state drives rendering and computed totals.

05 — Lab 6: CRUD and persistence

Create: register new software. Read: display records. Update: edit existing records. Delete: remove after confirmation. Bonus: instant search and filtering. Local Storage is the default mode; optional JSON Server implements GET, POST, PATCH and DELETE against db.json.

TESTING & RESULT

Verification and findings

VulnWatch — Lab Exercises 5 & 6

06 — Test coverage

Fifteen automated tests passed using the built-in Node test runner. The tests include input validation, duplicate checking, dynamic filtering, summary counts, scenario matches, weighted-priority boundaries, Local Storage persistence, complete CRUD, reset behavior, corrupt-data handling and REST connection failures. One test performs actual HTTP requests against a temporary in-process REST service.

Verified results

15 / 15	Node tests passed
14 / 14	Source JS / JSX files parsed successfully
PASS	CSS parsed without syntax errors
PASS	Real HTTP GET, POST, PATCH, DELETE integration test
PENDING	Vite production build and Chromium UI run (package installation blocked)

07 — Result

The source code delivers a reusable React frontend for managing a cybersecurity watchlist with persistent browser storage and an optional JSON Server backend. Sample risk scenarios respond to the monitored software list. A separate interactive lab demonstrates transparent severity / exploitation / exposure weighting.

08 — Limitations and responsible data labeling

The previous HTML referenced threat-feed endpoints but their backend code was not supplied. This application therefore labels all LAB-ALERT scenarios as educational examples; it does not claim real CVE, ThreatFox, GeoLite2, live-IP or reputation integration. The Geolocation API does not create fictitious threat markers. JSON Server is for local learning and has no authentication.

A complete Vite browser demonstration and screenshots must be captured after running npm install on an internet-connected machine. JavaScript/JSX syntax checks and the 15 automated source/data tests have been executed, but they do not replace a visual end-to-end test. The README includes instructions for capturing each CRUD screenshot.

09 — Conclusion

Labs 5 and 6 demonstrate the evolution of VulnWatch into a modular, responsive React application with useful state-driven forms, reusable components, persistence and REST-oriented CRUD. Future scope includes secure authentication, verified vulnerability feeds, accurate vendor/version matching and a production-grade backend.